



■ CS177 Bioinformatics: Software Development and Applications

Lecture 12: Pairwise Sequence Alignment

Jie Zheng (郑杰)

PhD, Associate Professor

School of Information Science and Technology (SIST), ShanghaiTech University

Spring, 2024





Learning objectives



- At the end of this lecture, you will be able to:
 - Understand the definition of pairwise sequence alignment and related terminology
 - Explain the purpose and derivation of scoring matrices, PAM and BLOSUM series
 - Describe the global algorithm of Needleman-Wunsch and the local algorithm of Smith-Waterman
 - Explain how BLAST works
 - Explain how deep learning can be applied to pairwise alignment





Introduction





Definition of pairwise alignment

- **Pairwise alignment:** The process of lining up two sequences to achieve maximal levels of identity (and conservation, in the case of amino acid sequences) for the purpose of assessing the degree of similarity and the possibility of homology
- Pairwise sequence alignment is the most fundamental operation in bioinformatics:
 - It is used to decide if two proteins (or genes) are related structurally or functionally
 - It is used to identify domains or motifs that are shared between proteins
 - It is the basis of BLAST searching
 - It is used in the analysis of genomes





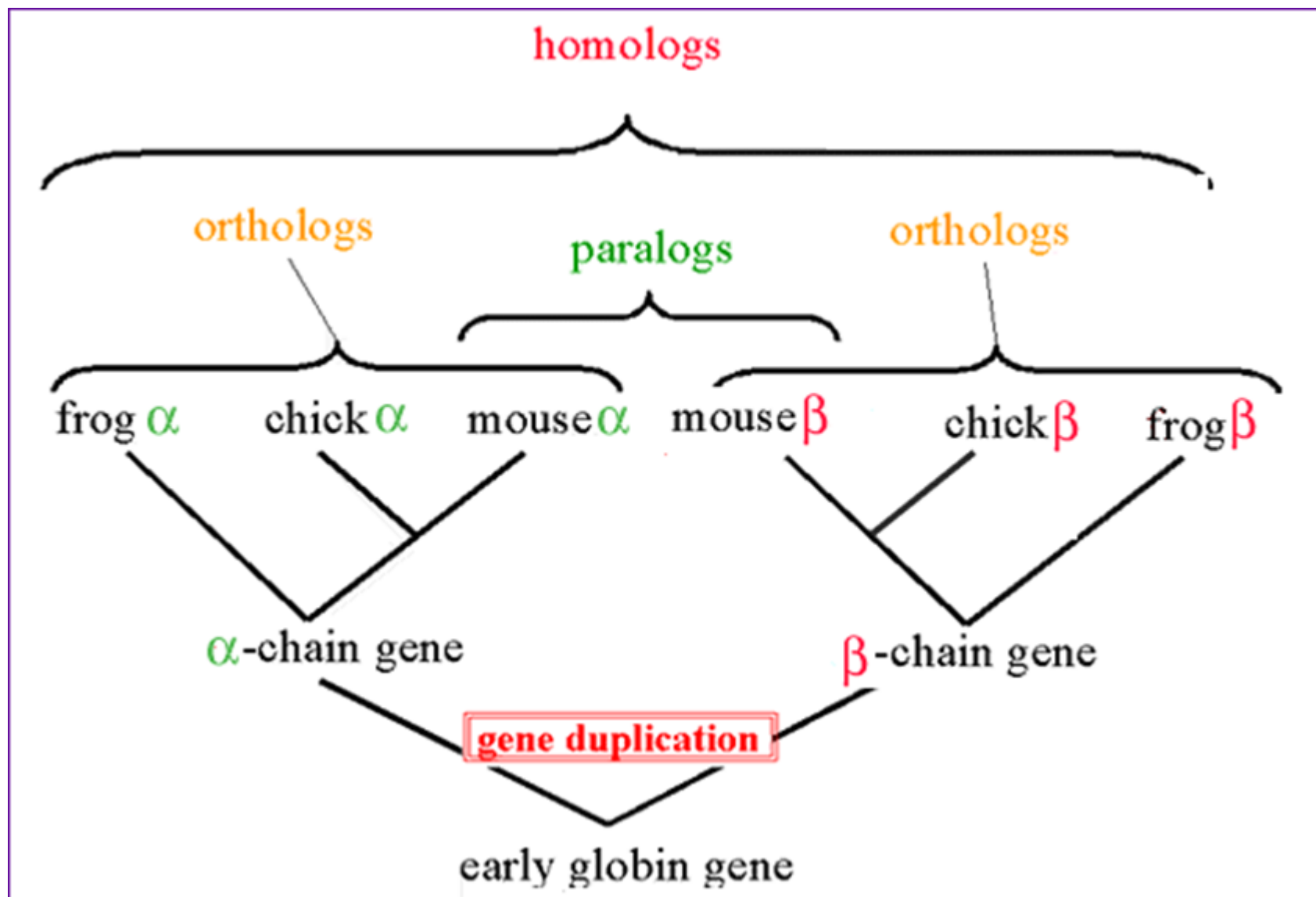
Terminology



- **Homology**: Similarity attributed to descent from a common ancestor
- There are two types of homology:
 - **Orthologs**: Homologous sequences in different species that arose from a common ancestral gene during speciation; may or may not be responsible for a similar function
 - **Paralogs**: Homologous sequences within a single species that arose by gene duplication
- **Identity**: The extent to which two (nucleotide or amino acid) sequences are invariant
- **Similarity**: The extent to which nucleotide or protein sequences are related. It is based upon identity plus conservation
- **Conservation**: Changes at a specific position of an amino acid or (less commonly, DNA) sequence that preserve the physico-chemical properties of the original residue



Orthologs and paralogs in a tree





Example of pairwise alignment



Pairwise alignment between human beta globin (query) and myoglobin (subject)

Score = 43.9 bits (102), Expect = 1e-09, Method: Composition-based stats.
Identities = 37/145 (25%), Positives = 57/145 (39%), Gaps = 2/145 (1%)

```

      ▼                               ▼
Query  4    LTPEEKSAVTALWGKVNVD--EVGGEALGRLLVVYPWTQRFFESFGDLSTPDVAMGNPKV  61
      → L+  E   V  +WGKV  D      G E L RL   +P T   F+ F  L + D +  +  +
Sbjct  3    LSDGEWQLVLNVWGKVEADIPGHGQEVLIIRLFKGHHPETLEKFDKFKHLKSEDEMKAEDL  62

Query  62    KAHGKKVLGAFSDGLAHLNLDNLKGTFFATLSELHCDKLHVDPENFRLLGNVLVCVLAHHFGK  121
      → K HG  VL A    L    + +      L++ H  K  +  +      +  ++ VL
Sbjct  63    KKHGATVLTALGGILKKKGHHEAEIKPLAQSHATKHKIPVKYLEFISECIIQVLQSKHPG  122

Query  122   EFTPPVQAAYQKVVAGVANALAHKY  146
      → +F    Q A  K +      +A  Y
Sbjct  123   DFGADAQGAMNKALELFRKDMASNY  147
```





Scoring an alignment

- For a set of aligned residues we assign scores based on
 - matches
 - mismatches
 - gap open penalties, and
 - gap extension penalties
- These scores add up to the total raw score

```
Score = 18.1 bits (35), Expect = 0.015, Method: Composition-based stats.
Identities = 11/24 (45%), Positives = 12/24 (50%), Gaps = 2/24 (8%)
```

```
Query 12 VTALWGKVNVD--EVGGEALGRLL 33
          V +WGKV D G E L RL
Sbjct 11 VLN VWGKVEADIPGHGQEV LIRLF 34
```

match	4	11	5	6	6	5	4	5	sum of matches: +60 (round up to +61)
			6	4				4	
mismatch	-1	1	0	-2	-2	-4	0		sum of mismatches: -13
	-2		0	-3	0				
gap open			-11						sum of gap penalties: -13
gap extend			-2						
									total raw score: 61 - 13 - 13 = 35



Gaps



- Positions at which a letter is paired with a null are called **gaps**
- Gap scores are typically negative
- Since a single mutational event may cause the insertion or deletion of more than one residue, the **presence** of a gap is ascribed more significance than the **length** of the gap. Thus, there are separate penalties for **gap creation** and **gap extension**





Scoring Matrices



■ Scoring function for DNA



- For DNA, since there are only 4 nucleotides, the score function is simple:
 - **BLAST matrix**
 - **Transition Transversion matrix**: give mild penalty for replacing purine by purine (A or G). Similar for replacing pyrimidine by pyrimidine (C or T)

	A	C	G	T
A	5	-4	-4	-4
C	-4	5	-4	-4
G	-4	-4	5	-4
T	-4	-4	-4	5

BLAST Matrix

	A	C	G	T
A	1	-5	-1	-5
C	-5	1	-5	-1
G	-1	-5	1	-5
T	-5	-1	-5	1

Transition Transversion Matrix





Scoring function for protein



- Commonly, it is devised based on two criteria:
 - Chemical/physical similarity
 - Observed substitution frequencies
- Using physical/chemical properties:
 - Idea: An amino acid is more likely to be substituted by another if they have **similar properties**
 - The score matrices can be derived based on hydrophobicity, charge, electronegativity, and size, etc.
 - E.g. we give a higher score for substituting nonpolar amino acid to another nonpolar amino acid





Scoring function for protein based on statistical model



- Most often used approaches
- Two popular matrices:
 - Point Accepted Mutation (PAM) matrix
 - BLOSUM
- Both methods define the score as the **log-odds ratio** between the observed substitution rate and the expected substitution rate in a random model





Pointed Accepted Mutation (PAM)



- PAM was developed by Margaret Dayhoff (1978)
- A **point mutation** means substituting one residue by another
- It is called an **accepted point mutation** if the mutation does not change the protein's function or is not fatal
- Two sequences S_1 and S_2 are said to be **1 PAM diverged**, if a series of accepted point mutations can convert S_1 to S_2 with an average of **1 accepted point mutation per 100 residues**



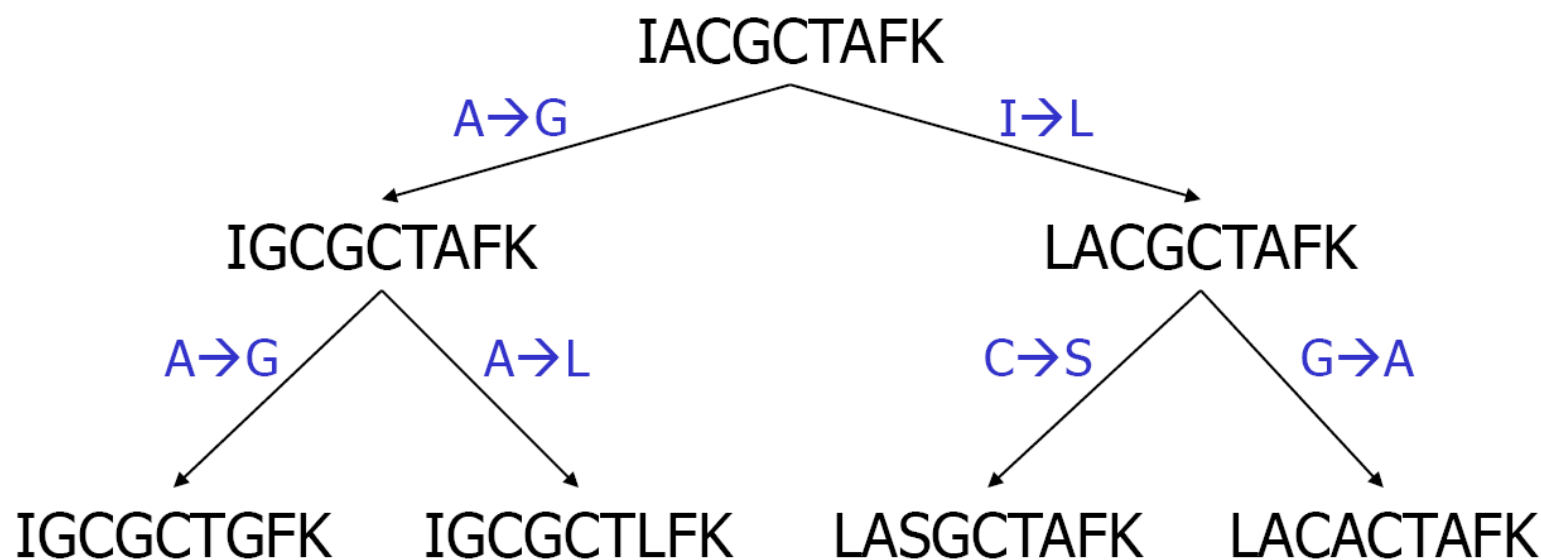


Construction of PAM matrix



- Ungapped alignment is constructed by high-similarity amino acid sequences (usually > 85%)
- Below is a simplified global multiple alignment of some highly similar amino acid sequences (without gap)
- A phylogenetic tree can be built for the sequences

IACGCTAFK
IGCGCTAFK
LACGCTAFK
IGCGCTGFK
IGCGCTLFK
LASGCTAFK
LACACTAFK





PAM-1 matrix



$$\delta(a, b) = \log_{10} \frac{O_{a,b}}{E_{a,b}}$$

where $O_{a,b}$ and $E_{a,b}$ are the observed frequency and the expected frequency of substituting a by b or b by a

- Since PAM-1 assumes **1 mutation per 100 residues**, $O_{a,a} = 99/100$
- For $a \neq b$, $O_{a,b} = F_{a,b} / (100 \sum_x \sum_y F_{x,y})$ where $F_{a,b}$ is the frequency of substituting a by b or b by a
- $E_{a,b} = f_a * f_b$ where f_a is the number of a divided by the total number of residues
- E.g., $F_{A,G} = 3$, $F_{A,L} = 1$, $f_A = f_G = 10/63$
- $O_{A,G} = 3/(100*2*6) = 0.0025$
- $E_{A,G} = (10/63)(10/63) = 0.0252$
- $\delta(A, G) = \log (0.0025/0.0252) = \log (0.0992063) = -1.00346$
- The entries in the matrix are rounded to integers for computational efficiency





PAM-n matrix



- Let $M_{a,b}$ be the probability that a is mutated to b , which equals $O_{a,b}/f_a$
- In general, PAM-n matrix is created by extrapolating the PAM-1 matrix
- $M^n(a, b)$ is the probability that a is mutated to b after n mutations
- Then, the (a, b) entry of the PAM-n matrix is

$$\log(f_a M^n(a, b)/(f_a f_b)) = \log(M^n(a, b)/f_b)$$





BLOSUM (BLOck SUBstitution Matrix)



- PAM did not work well for aligning evolutionarily divergent sequences since the matrix is generated by extrapolation
- Steven Henikoff and Jorja Henikoff (1992) proposed BLOSUM
- Unlike PAM, BLOSUM matrix is constructed directly from the observed local alignments (instead of extrapolation)
- In BLOSUM, the input is the set of multiple alignments for nonredundant groups of protein families
 - Based on PROTOMAT (the block-making procedure of the PROSITE database), blocks of nongapped local alignments are derived
 - Each block represents a conserved region of a protein family





Scoring function of BLOSUM



- From all blocks, we count the frequency f_a for each amino acid residue a
- For any two amino acid residues a and b , we count the frequency p_{ab} of the aligned pair of a and b
- The alignment score is $\delta(a, b) = 1/\lambda \ln O_{ab}/(f_a f_b)$, where
 - O_{ab} is the probability that a and b are observed to align together
 - f_a and f_b are the frequencies of residues a and b
 - λ is a normalization constant
- Example: In the block on the right side, there are $7 \times 9 = 63$ residues, including 9 A' s and 10 G' s. Hence, $f_A = 9/63$, $f_G = 10/63$
- There are $9 \binom{7}{2} = 189$ aligned residue pairs, including 23 (A, G) pairs. Hence, $O_{AG} = 23/189$. With $\lambda = 0.347$, $\delta(A, G) = 4.54$

```
ACGCTAFKI
GCGCTAFKI
ACGCTAFKL
GCGCTGFKI
GCGCTLFKI
ASGCTAFKL
ACACTAFKL
```





BLOSUM 62



- BLOSUM p matrix is created by merging sequences with no less than p% identity
- BLOSUM62 is a matrix calculated from comparison of sequences with no less than 62% identity
- BLOSUM62 is the default matrix in NCBI BLAST 2.0
- The entries in the matrix are rounded to integers

A	4																			
R	-1	5																		
N	-2	0	6																	
D	-2	-2	1	6																
C	0	-3	-3	-3	9															
Q	-1	1	0	0	-3	5														
E	-1	0	0	2	-4	2	5													
G	0	-2	0	-1	-3	-2	-2	6												
H	-2	0	1	-1	-3	0	0	-2	8											
I	-1	-3	-3	-3	-1	-3	-3	-4	-3	4										
L	-1	-2	-3	-4	-1	-2	-3	-4	-3	2	4									
K	-1	2	0	-1	-1	1	1	-2	-1	-3	-2	5								
M	-1	-2	-2	-3	-1	0	-2	-3	-2	1	2	-1	5							
F	-2	-3	-3	-3	-2	-3	-3	-3	-1	0	0	-3	0	6						
P	-1	-2	-2	-1	-3	-1	-1	-2	-2	-3	-3	-1	-2	-4	7					
S	1	-1	1	0	-1	0	0	0	-1	-2	-2	0	-1	-2	-1	4				
T	0	-1	0	-1	-1	-1	-1	-2	-2	-1	-1	-1	-1	-2	-1	1	5			
W	-3	-3	-4	-4	-2	-2	-3	-2	-2	-3	-2	-3	-1	1	-4	-3	-2	11		
Y	-2	-2	-2	-3	-2	-1	-2	-3	2	-1	-1	-2	-1	3	-3	-2	-2	2	7	
V	0	-3	-3	-3	-1	-2	-2	-3	-3	3	1	-2	1	-1	-2	-2	0	-3	-1	4
	A	R	N	D	C	Q	E	G	H	I	L	K	M	F	P	S	T	W	Y	V

BLOSUM62 scoring matrix





Global Alignment: Needleman-Wunsch Algorithm





Global and local sequence alignment



- We will first explore the global alignment algorithm of Needleman and Wunsch (1970)
- Then, we will describe the local alignment algorithm of Smith and Waterman (1981)
- Finally, we will introduce BLAST, a heuristic version of Smith-Waterman





Global alignment: Needleman-Wunsch algorithm

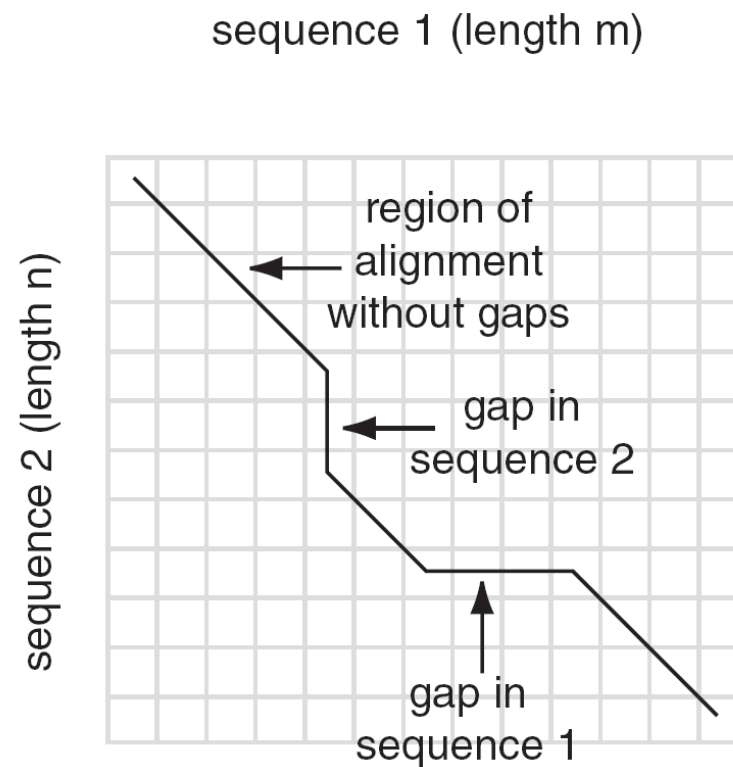


- Two sequences can be compared in a matrix along the x- and y-axes
 - If they are identical, a path along a diagonal can be drawn
- Find the optimal subpaths, and add them up to achieve the best score. This involves
 - adding gaps when needed
 - allowing for conservative substitutions
 - choosing a scoring system (simple or complicated)
- N-W algorithm is guaranteed to find optimal alignment(s)





4 possible outcomes in aligning 2 sequences

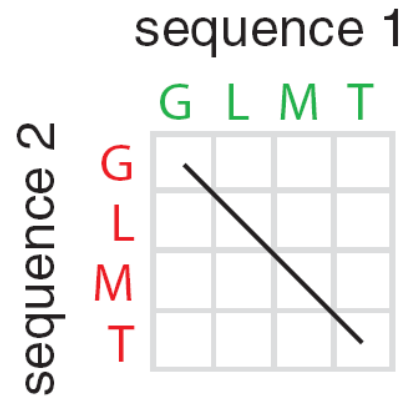


- [1] identity (stay along a diagonal)
- [2] mismatch (stay along a diagonal)
- [3] gap in one sequence (move vertically)
- [4] gap in the other sequence (move horizontally)

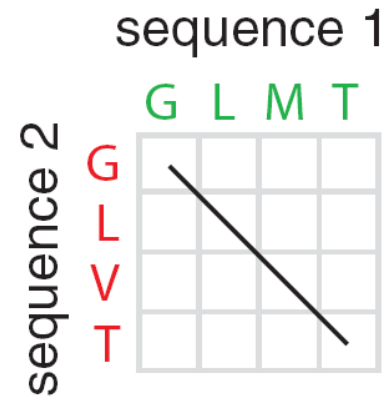




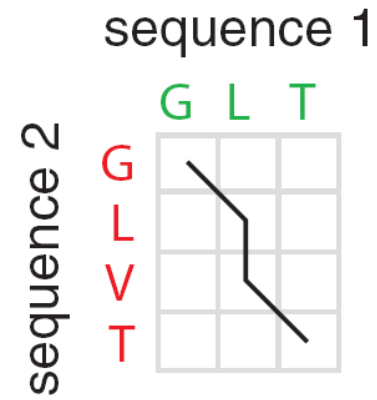
4 possible outcomes in aligning 2 sequences



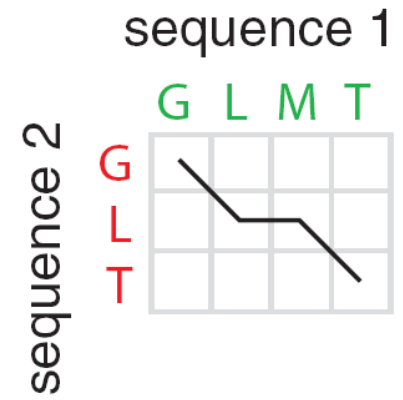
1 GLMT
2 GLMT



1 GLMT
2 GLVT



1 GL-T
2 GLVT



1 GLMT
2 GL-T





- We define an overall score that maximizes cumulative scores at each position of the pairwise alignment, allowing for substitutions and gaps in either sequence

$$\text{Score} = \text{Max} \begin{cases} F(i-1, j-1) + s(x_i, y_i) \\ F(i-1, j) - \text{gap penalty} \\ F(i, j-1) - \text{gap penalty} \end{cases}$$

Score (this example) = +1 (match)
-2 (mismatch)
-2 (gap penalty)

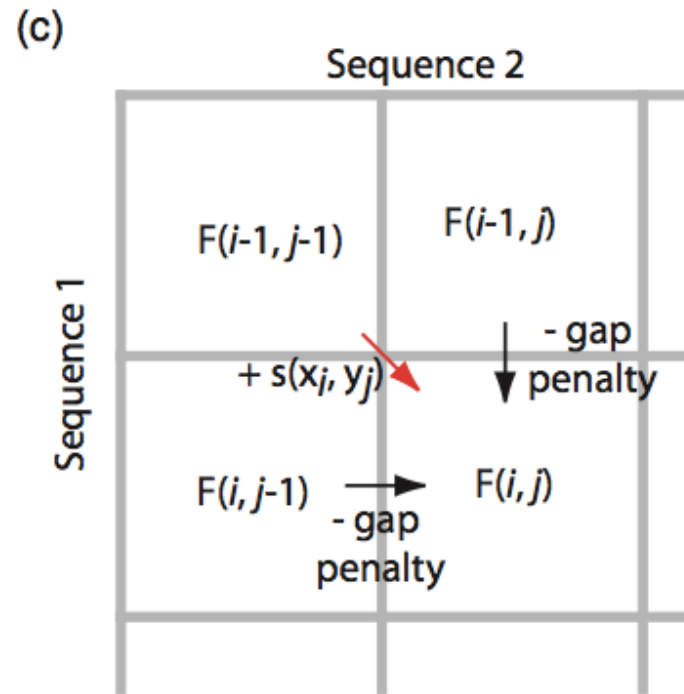




Decision in one step



- To decide how to align sequences 1 and 2 in the box at lower right, decide what the scores are
 - beginning at **upper left** (not requiring a gap), or
 - beginning from the **left** or **top** (each requiring a gap penalty)

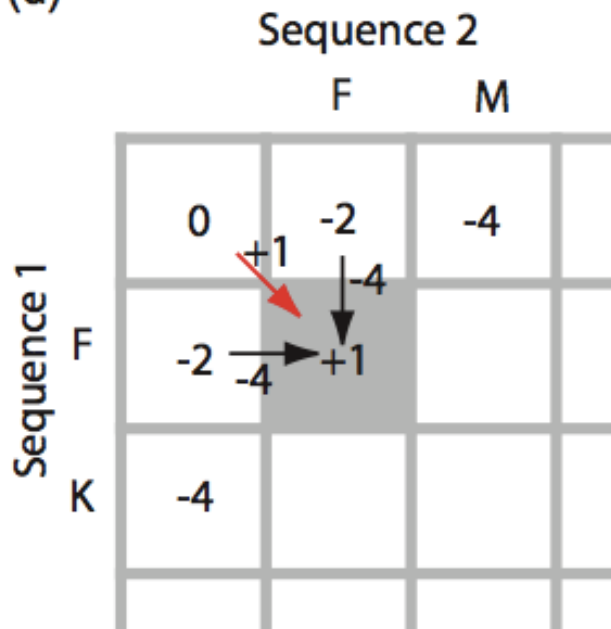




Two example steps

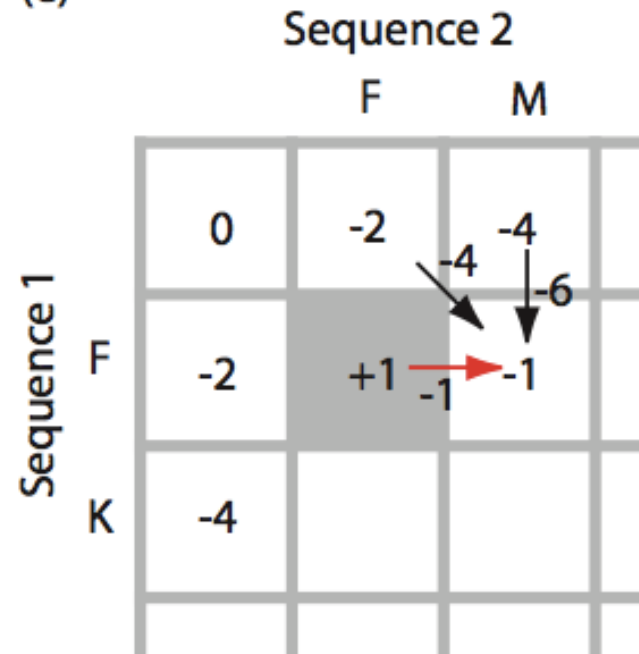


(d)



Here the best score involves +1 (proceed from upper left to gray, lower right square). If we instead select an alignment involving a gap the score would be worse (-4)

(e)



Proceed to calculate the optimal score for the next position





Continue filling in the matrix



(f)

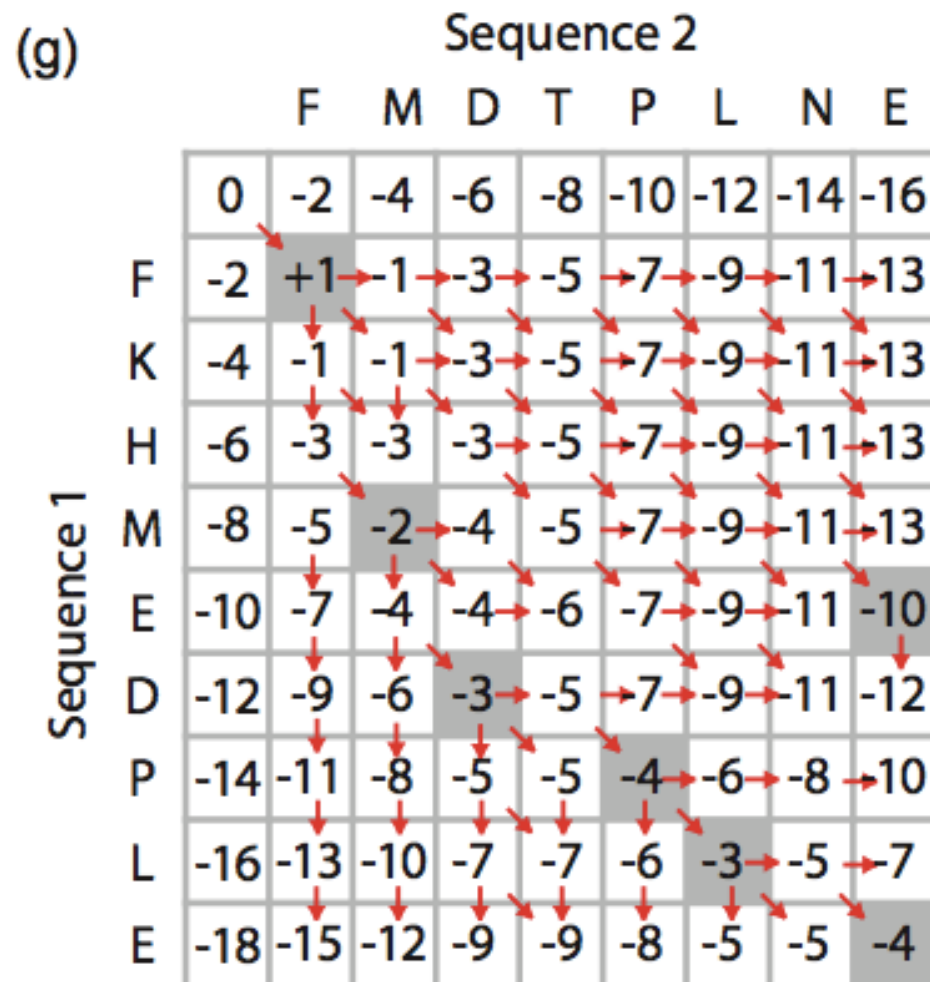
Sequence 2

		F	M	D	T	P	L	N	E	
Sequence 1		0	-2	-4	-6	-8	-10	-12	-14	-16
	F	-2	+1	-1	-3	-5	-7	-9	-11	-13
	K	-4	-1							
	H	-6								
	M	-8								
	E	-10								
	D	-12								
	P	-14								
	L	-16								
	E	-18								





Complete filling in the matrix

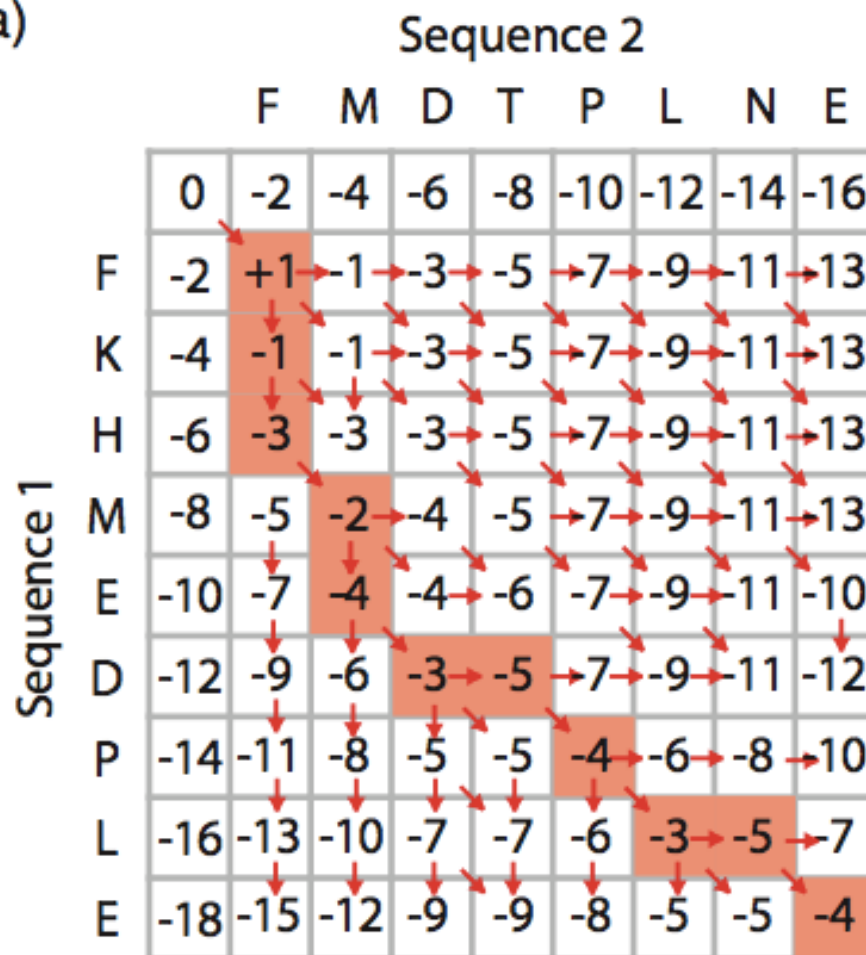


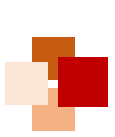


Optimal path (best score) highlighted



(a)





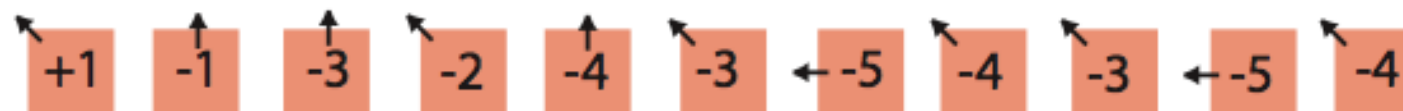
Traceback of the optimal path



(b)

Sequence 2

		F	M	D	T	P	L	N	E	
Sequence 1		0	-2	-4	-6	-8	-10	-12	-14	-16
	F	-2	+1	-1	-3	-5	-7	-9	-11	-13
	K	-4	-1	-1	-3	-5	-7	-9	-11	-13
	H	-6	-3	-3	-3	-5	-7	-9	-11	-13
	M	-8	-5	-2	-4	-5	-7	-9	-11	-13
	E	-10	-7	-4	-4	-6	-7	-9	-11	-10
	D	-12	-9	-6	-3	-5	-7	-9	-11	-12
	P	-14	-11	-8	-5	-5	-4	-6	-8	-10
	L	-16	-13	-10	-7	-7	-6	-3	-5	-7
	E	-18	-15	-12	-9	-9	-8	-5	-5	-4



Sequence 1	F	K	H	M	E	D	-	P	L	-	E
Sequence 2	F	-	-	M	-	D	T	P	L	N	E





Needleman-Wunsch: dynamic programming



- N-W is guaranteed to find optimal alignments, although the algorithm does not search all possible alignments
- It is an example of a **dynamic programming** algorithm:
 - An optimal path (alignment) is identified by incrementally extending optimal subpaths
 - Thus, a series of decisions is made at each step of the alignment to find the pair of residues with the best score





Local Alignment: Smith-Waterman Algorithm





Global alignment versus local alignment



- Global alignment (Needleman-Wunsch) extends from one end of each sequence to the other
- Local alignment finds optimally matching regions within two sequences ("subsequences")
- Local alignment is almost always used for database searches such as BLAST. It is useful to find domains (or limited regions of homology) within sequences
- Smith and Waterman (1981) solved the problem of performing optimal local sequence alignment. Other methods (BLAST, FASTA) are faster but less thorough





Smith-Waterman algorithm



- Set up a matrix between two proteins (size $m+1, n+1$)
- No values in the scoring matrix can be negative: $S \geq 0$
- The score in each cell is the maximum of four values:
 - [1] $s(i-1, j-1)$ + the new score at $[i, j]$ (a match or mismatch)
 - [2] $s(i, j-1)$ – gap penalty
 - [3] $s(i-1, j)$ – gap penalty
 - [4] zero ← this is not in Needleman-Wunsch





Example of S-W



(a)

(a)		Sequence 1													
		C	A	G	C	C	U	C	G	C	U	U	A	G	
Sequence 2		0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	
	A	0.0	0.0	1.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	1.0	0.0
	A	0.0	0.0	1.0	0.7	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	1.0	0.7
	U	0.0	0.0	0.0	0.7	0.3	0.0	1.0	0.0	0.0	0.0	1.0	1.0	0.0	0.7
	G	0.0	0.0	0.0	1.0	0.3	0.0	0.0	0.7	1.0	0.0	0.0	0.7	0.7	1.0
	C	0.0	1.0	0.0	0.0	2.0	1.3	0.3	1.0	0.3	2.0	0.7	0.3	0.3	0.3
	C	0.0	1.0	0.7	0.0	1.0	3.0	1.7	1.3	1.0	1.3	1.7	0.3	0.0	0.0
	A	0.0	0.0	2.0	0.7	0.3	1.7	2.7	1.3	1.0	0.7	1.0	1.3	1.3	0.0
	U	0.0	0.0	0.7	1.7	0.3	1.3	2.7	2.3	1.0	0.7	1.7	2.0	1.0	1.0
	U	0.0	0.0	0.3	0.3	1.3	1.0	2.3	2.3	2.0	0.7	1.7	2.7	1.7	1.0
	G	0.0	0.0	0.0	1.3	0.0	1.0	1.0	2.0	3.3	2.0	1.7	1.3	2.3	2.7
	A	0.0	0.0	1.0	0.0	1.0	0.3	0.7	0.7	2.0	3.0	1.7	1.3	2.3	2.0
	C	0.0	1.0	0.0	0.7	1.0	2.0	0.7	1.7	1.7	3.0	2.7	1.3	1.0	2.0
	G	0.0	0.0	0.7	1.0	0.3	0.7	1.7	0.3	2.7	1.7	2.7	2.3	1.0	2.0
	G	0.0	0.0	0.0	1.7	0.7	0.3	0.3	1.3	1.3	2.3	1.3	2.3	2.0	2.0

(b)

sequence 1 GCC-UCG
sequence 2 GCCAUUG

(c)

sequence 1 CA-GCC-UCGCUUAG
sequence 2 AAUGCCAUUGACG-G





Where to use Smith-Waterman



[1] Galaxy offers “needle” and “water” EMBOSS programs

[2] EBI offers needle and water: <http://www.ebi.ac.uk/Tools/psa/>

[3] Try using SSEARCH to perform a rigorous Smith-Waterman local alignment: <http://fasta.bioch.virginia.edu/>

[4] Next-generation sequence aligners incorporate Smith-Waterman in some specialized steps





Rapid, heuristic versions of Smith-Waterman: FASTA and BLAST



- Smith-Waterman is very rigorous and it is guaranteed to find an optimal alignment
- But Smith-Waterman is slow. It requires computer space and time proportional to the product of the two sequences being aligned (or the product of a query against an entire database)
- Gotoh (1982) and Myers and Miller (1988) improved the algorithms so that both global and local alignment require less time and space
- FASTA and BLAST provide rapid alternatives to Smith-Waterman





BLAST





BLAST



上海科技大学
ShanghaiTech University

- BLAST (Basic Local Alignment Search Tool) allows rapid sequence comparison of a query sequence against a database
- The BLAST algorithm is **fast**, **accurate**, and **accessible** both via the web and the command line

query: FASTA format or accession

1

database

2

Entrez query

3

algorithm

4

parameters

5

BLAST® Basic Local Alignment Search Tool

Home Recent Results Saved Strategies Help

My NCBI Welcome pepsner. [Sign Out]

NCBI/ BLAST/ blastp suite Standard Protein BLAST

blastn blastp blastx tblastn tblastx

BLASTP programs search protein databases using a protein query. [more...](#) [Reset page](#) [Bookmark](#)

Enter Query Sequence

Enter accession number(s), gi(s), or FASTA sequence(s) [Clear](#) Query subrange [From](#) [To](#)

>gi|4504349|ref|NP_000509.1| hemoglobin subunit beta [Homo sapiens]
MVHLTFEEKSAVTAIWGKYNVDEVGGEALGRLVVVFWIQRFESFGDLSTPDAMGNPKVKAR
GKKVLGAFSDGLAHLNLTGTFATLSELHCCKLHVDPENFLLGNVLCVLAHHEGKEFTIPVQ
AAYQKVAGVANALAHKYH

Or, upload file [Browse...](#)

Job Title
Enter a descriptive title for your BLAST search

☐ Align two or more sequences

Choose Search Set

Database [?](#)

Organism Optional ☐ Exclude [+](#)
Enter organism common name, binomial, or tax id. Only 20 top taxa will be shown.

Exclude Optional ☐ Models (XM/XP) ☐ Uncultured/environmental sample sequences

Entrez Query Optional
Enter an Entrez query to limit search

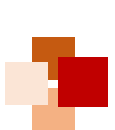
Program Selection

Algorithm ☒ blastp (protein-protein BLAST)
☐ PSI-BLAST (Position-Specific Iterated BLAST)
☐ PHI-BLAST (Pattern Hit Initiated BLAST)
☐ DELTA-BLAST (Domain Enhanced Lookup Time Accelerated BLAST)
[Choose a BLAST algorithm](#)

BLAST Search database Reference proteins (refseq_protein) using Blastp (protein-protein BLAST)
☐ Show results in a new window

[Algorithm parameters](#) Note: Parameter values that differ from the default are highlighted

立志成才 报效国家



How a BLAST search works



- “The central idea of the BLAST algorithm is to confine attention to segment pairs that contain a word pair of length w with a score of at least T . ”
-- Altschul et al. (1990)
- Three phases:
 - (1) compile a list of word pairs above threshold T
 - (2) scan the database for matches and extend
 - (3) traceback to generate gapped alignment





■ Phase 1: compile a list of words ($w=3$)

- Compile a list of word pairs (e.g. $w=3$) with scores (based on BLOSUM) above threshold T
- Example: For a human RBP query ...FS**GTW**YA... (query word is in **green**)
- A list of words ($w=3$) is:
FSG SGT GTW TWY WYA
YSG TGT ATW SWY WFA
FTG SVT GSW TWF WYS
...





Phase 1: compile a list of words ($w=3$)



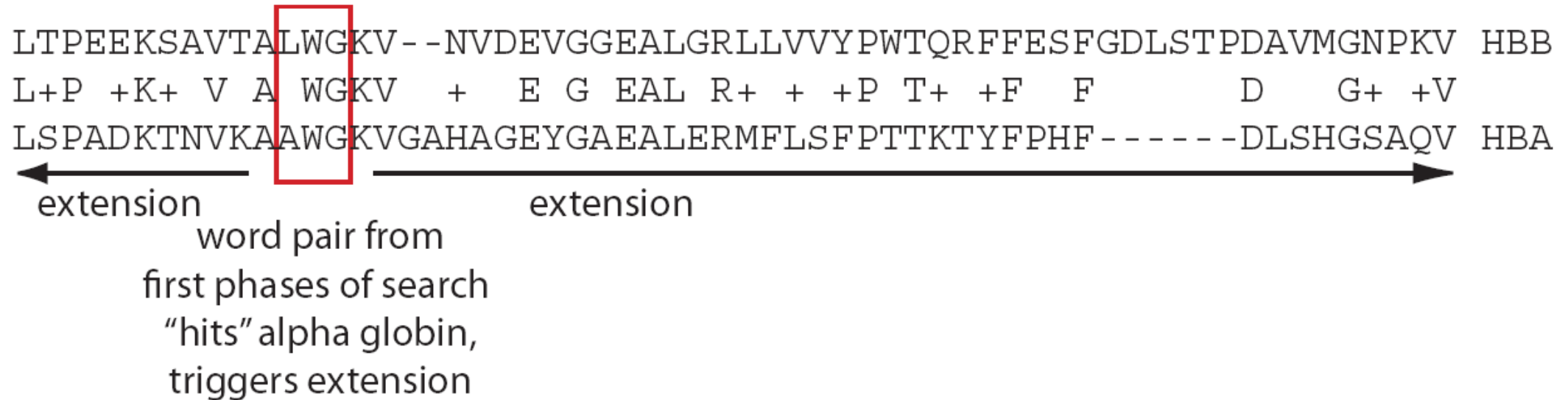
neighborhood word hits > threshold	GTW	6+5+11	22
	GSW	6+1+11	18
	ATW	0+5+11	16
	NTW	0+5+11	16
	GTY	6+5+2	13
(T=11)			
neighborhood word hits < below threshold	GNW		10
	GAW		9





Phase 2: scan the database for matches and extend

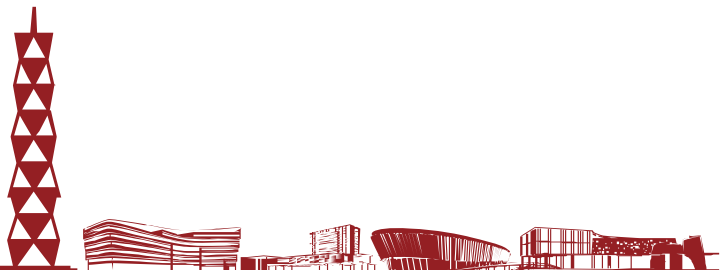
- Select all the words above threshold T (LWG, IWG, ...)
- Scan the database for entries ("hits") that match the compiled list
- Create a hash table indexed with the locations of all the hits for each word
- Perform gap free extensions
- Perform gapped extensions





Phase 3: Traceback to generate gapped alignment

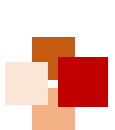
- Calculate locations of insertions, deletions, and matches (for alignments saved in Phase 2)
- Apply composition-based statistics (for BLASTP, TBLASTN)
- Generate gapped alignment





DEDAL: A Case of Deep Learning for Pairwise Alignment of Proteins





Two weaknesses of Smith-Waterman algorithm



- **Issue 1:** It lacks flexibility in the formulation of the score of a path
 - By enforcing that the cost of opening or extending a gap is the **same** wherever in the sequence
 - By enforcing that the substitution score of aligning two positions **only** depends on the amino acids at those positions
 - Flexible parameterizations may give more opportunities to optimize a relevant score
- **Issue 2:** The best alignment returned by the algorithm strongly depends on the choice of the parameters
 - There are no universally “good” substitution scores that lead to good alignments for all protein pairs
 - How to optimally choose adequate scoring parameters for a given pair of proteins



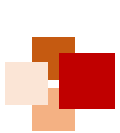


DEDAL (deep embedding and differentiable alignment)

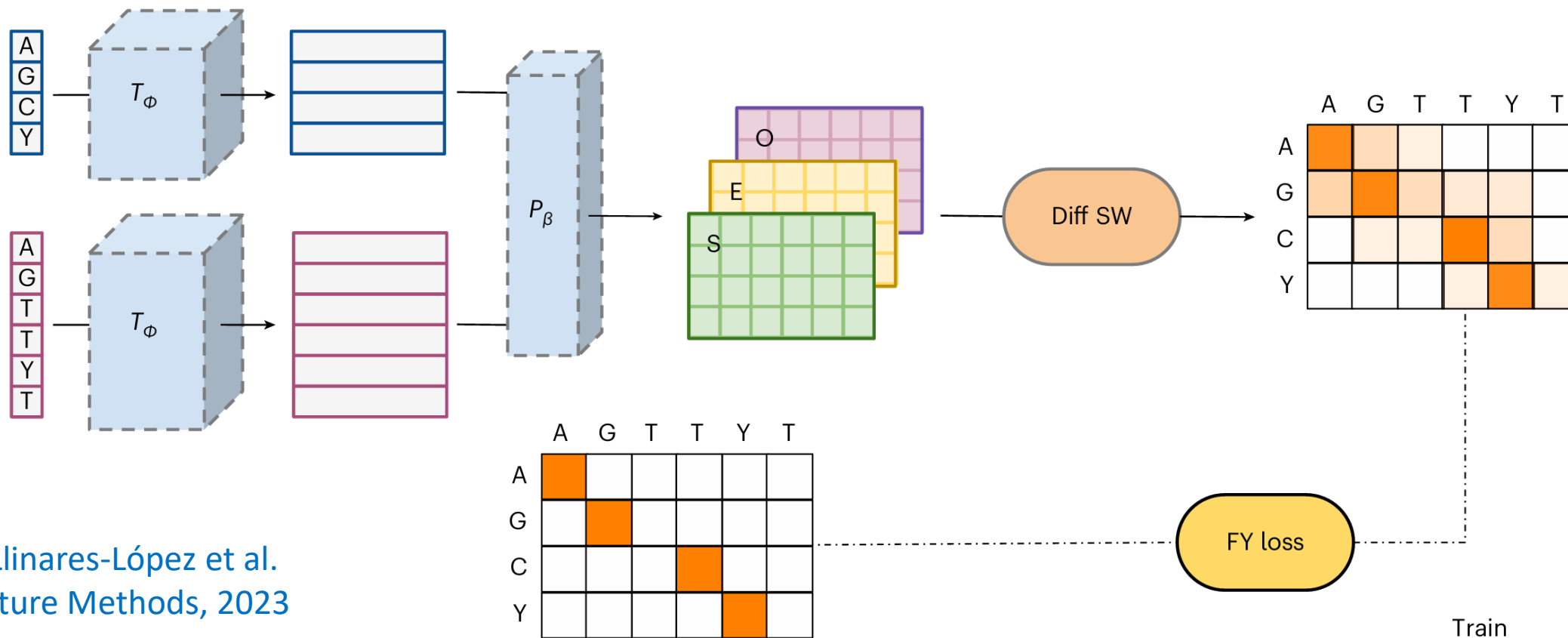


- Main idea:
 - DEDAL builds on top of the standard Smith-Waterman (SW) algorithm
 - Provides a flexible parameterization of the scoring function used by SW that adapts to each sequence pair and each position in each sequence
 - The parameterization is automatically learned during a training phase
- Training data:
 - A set of sequence pairs with known alignments
 - A large set of raw protein sequences
- Two pillars:
 - Deep learning language models, which
 - (1) embed discrete sequences in a continuous space;
 - (2) are automatically trained on many raw sequences
 - A parameterization of the SW algorithm (gap and substitution parameters) as a function of the continuous embedding





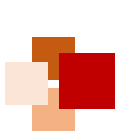
Training



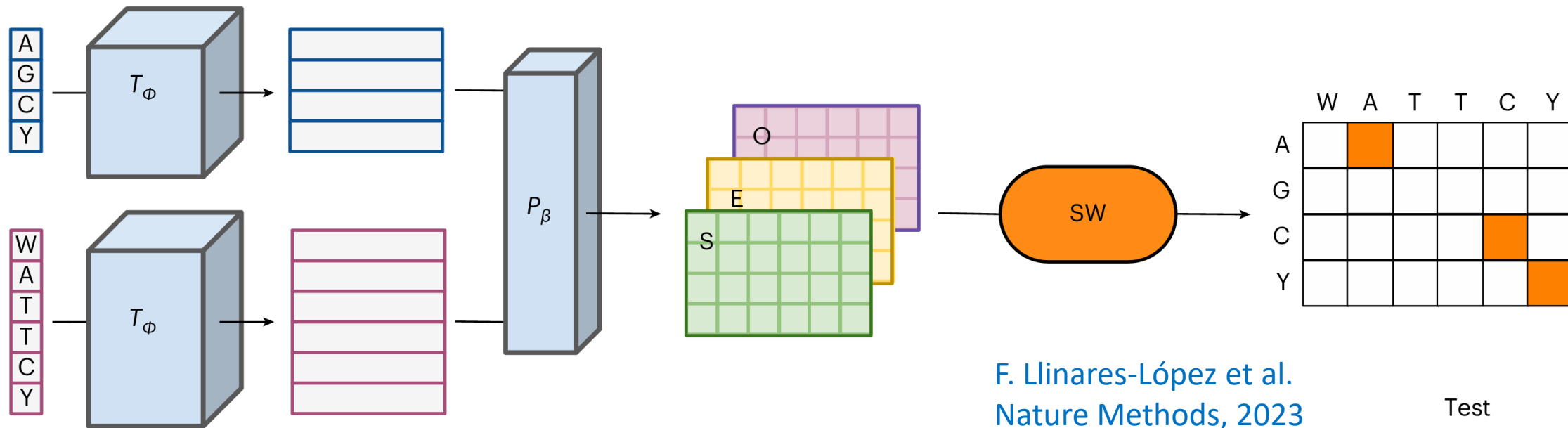
F. Llinares-López et al.
Nature Methods, 2023

- **Transformer encoder network T_ϕ** : To obtain a continuous representation of each input sequence
- **Parameterizer P_β** : transforms the continuous representations into parameter matrices used by SW
- The parameters Φ and β are learned from (1) a large corpus of raw sequences, and (2) pairs of sequences with known alignment, by end-to-end optimization





Testing



- Given two protein sequences, the transformer encoder T_ϕ is used to obtain a continuous representation (embedding) of each sequence
- For each pair of residues from the sequence pair, the parameterizer function P_β is used to compute a substitution score (S) and gap open (O) and gap extend (E) penalties from the continuous representations
- The standard SW algorithm is used to compute an optimal alignment and score it



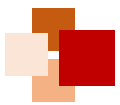


Recap



- In this lecture, we have learned:
 - That pairwise sequence alignment is a fundamental problem in bioinformatics
 - PAM and BLOSUM series of scoring matrices were inferred from probability models and sequence alignment data
 - Needleman-Wunsch (global) and Smith-Waterman (local) algorithms are based on dynamical programming
 - BLAST is a rapid, heuristic alternative of Smith-Waterman (SW)
 - A case of deep learning for pairwise alignment: DEDAL (deep embedding and differential alignment) uses Transformer encoder and flexible differentiable variant of the SW algorithm





References



- R. Durbin, S. Eddy, A. Krogh, G. Mitchison (1998). "Biological sequence analysis: Probabilistic models of proteins and nucleic acids" , Chapter 2 "Pairwise alignment" , Cambridge University Press
- Jonathan Pevsner (2015). "Bioinformatics and Functional Genomics " (3rd Edition), Chapter 3 "Pairwise Sequence Alignment" and Chapter 4 "Basic Local Alignment Search Tool (BLAST)"
- Needleman, Saul B. & Wunsch, Christian D. (1970). "A general method applicable to the search for similarities in the amino acid sequence of two proteins" . *Journal of Molecular Biology*. 48(3): 443–453
- Smith, Temple F. & Waterman, Michael S. (1981). "Identification of common molecular subsequences" . *Journal of Molecular Biology*. 147(1): 195–197
- S. Altschul, W. Gish, W. Miller, E. Myers, D. Lipman (1990). "Basic local alignment search tool" . *Journal of Molecular Biology*. 215(3): 403–410
- Karlin, S. & Altschul, S. F. (1990). "Methods for assessing the statistical significance of molecular sequence features by using general scoring schemes" . *Proc. Nat. Acad. Sci., U.S.A.* 87, 2264–2268
- F. Llinares-López et al. (2023). "Deep embedding and alignment of protein sequences" , *Nature Methods*, Vol. 20, pp. 104-111





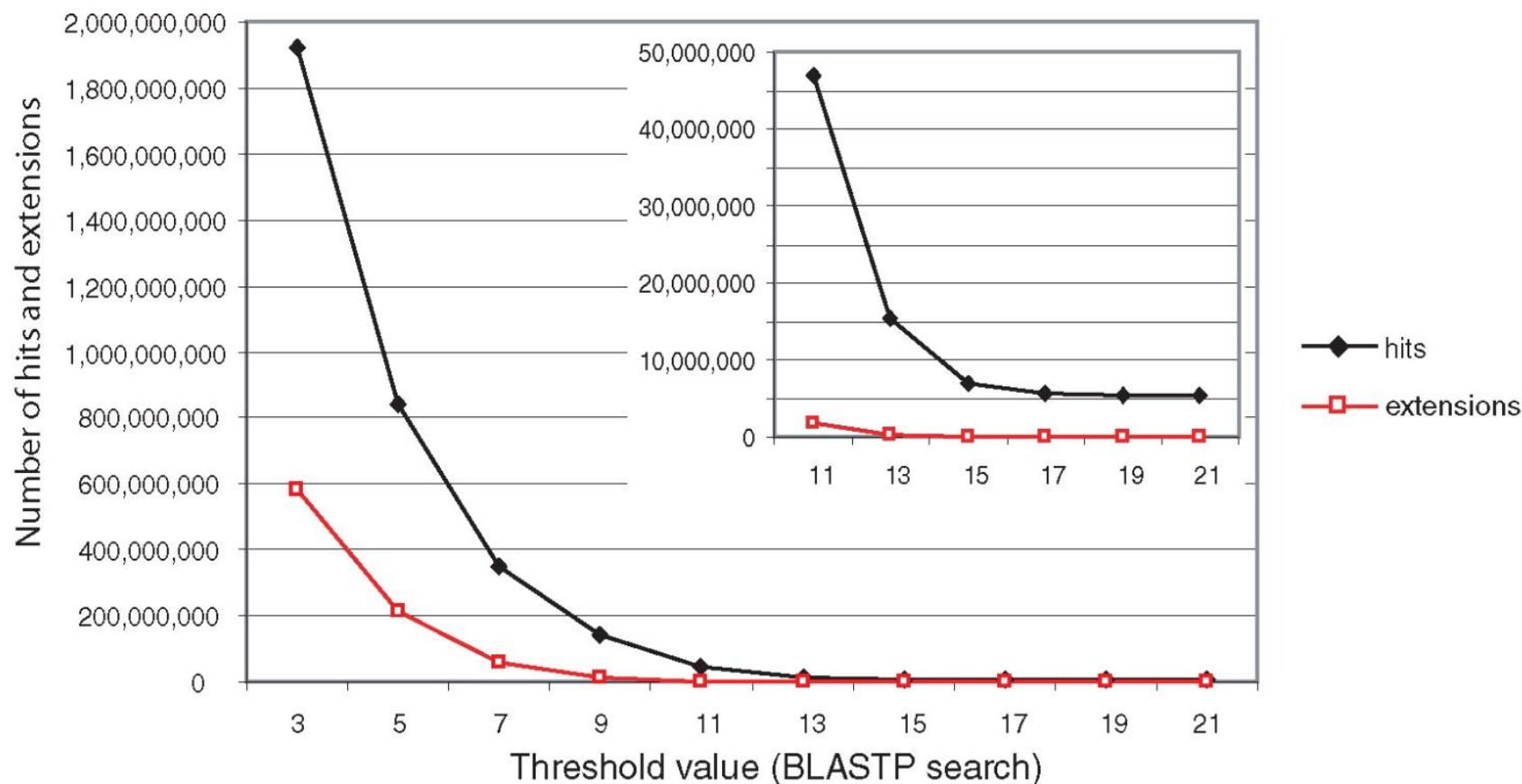
Appendix: Evaluation of BLAST Search Result



Parameters of BLAST



- The threshold value T can be modified (the default value for BLASTP is 11)
- The word size for BLASTN is typically 7, 11 or 15
- Changing word sizes is like changing thresholds, e.g. $w = 15$ gives fewer matches, but is faster than $w = 11$ or $w = 7$



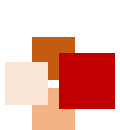


From raw scores to bit scores



- There are two kinds of scores:
 - **raw scores**: calculated from a substitution matrix
 - **bit scores**: normalized scores
- Bit scores are comparable between different searches because they are normalized to account for the use of different **scoring matrices** and different database **sizes**
 - $S' = \text{bit score} = (\lambda S - \ln K) / \ln 2$
 - The **E value** corresponding to a given bit score is:
$$E = mn 2^{-S'}$$
 - Bit scores allow you to compare results between different database searches, even using different scoring matrices





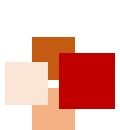
E-values and *p*-values (1)



- The **expect value** E is the number of alignments with scores greater than or equal to score S that are expected to occur by chance in a database search
- A ***p*-value** is a different way of representing the significance of an alignment

$$p = 1 - e^{-E}$$





E-values and *p*-values (2)



E values of about 1 to 10 are far easier to interpret than corresponding *p* values

Very small *E* values are very similar to *p* values

<u><i>E</i></u>	<u><i>p</i></u>
10	0.99995460
5	0.99326205
2	0.86466472
1	0.63212056
0.1	0.09516258 (about 0.1)
0.05	0.04877058 (about 0.05)
0.001	0.00099950 (about 0.001)
0.0001	0.00010000

- *E*-values are comparable to *p*-values, and are designed to be more convenient to interpret





Statistical significance of alignment



- It is important to assess the **statistical significance** of search results
- For global alignments, the statistics are poorly understood
- For local alignments (including BLAST search results), the statistics are well understood. The scores follow an **extreme value distribution (EVD)** rather than a normal distribution

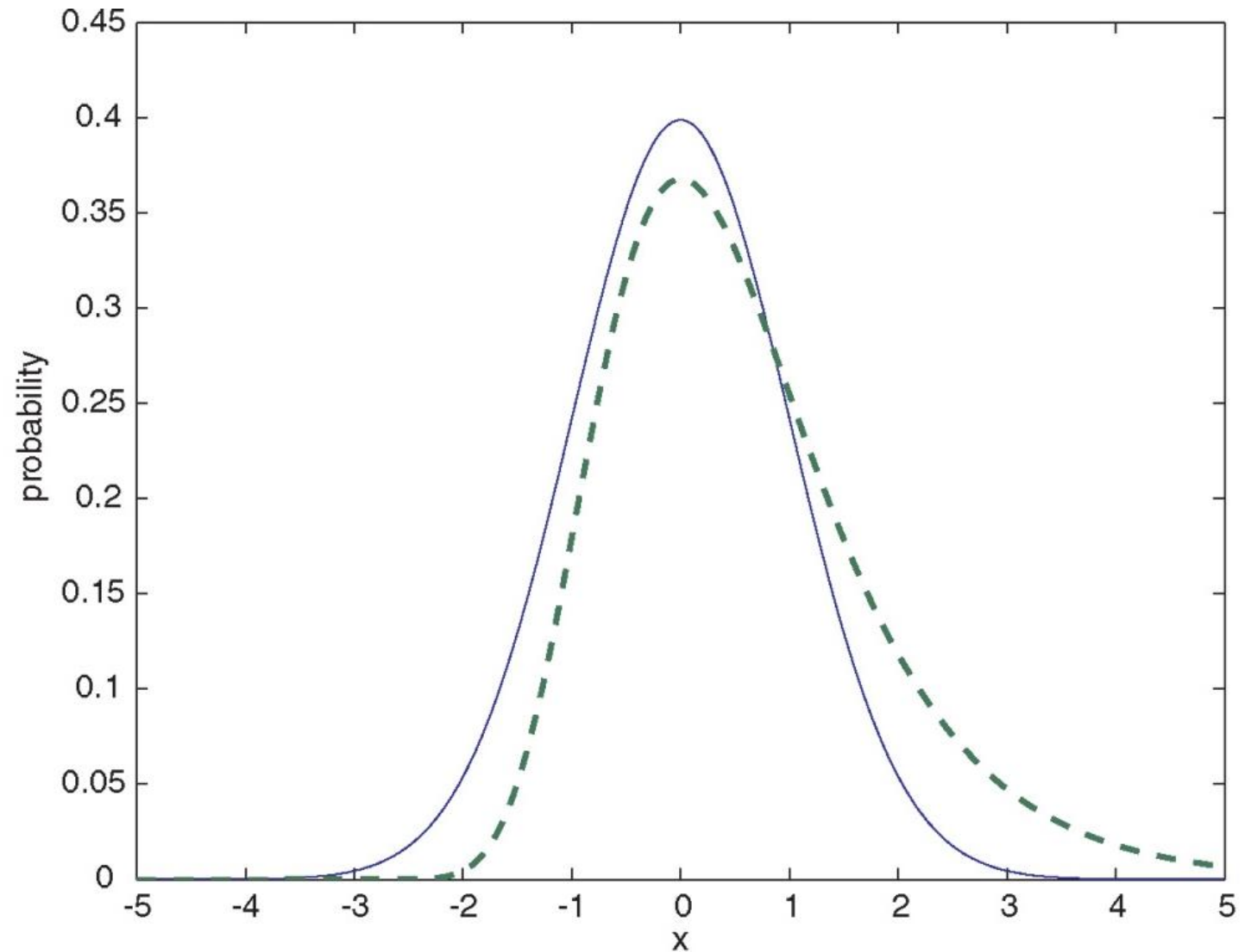


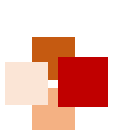


Normal distribution vs EVD



- Normal distribution (solid line) compared to extreme value distribution (dashed line)
- Note: EVD skewing to the right





E-value of BLAST search (1)



- The **expect value** E is the number of alignments with scores greater than or equal to score S that are expected to occur by chance in a database search
- The key equation describing an E value is:

$$E = Kmn e^{-\lambda S}$$





***E*-value of BLAST search (2)**



$$E = Kmn e^{-\lambda S}$$

- This equation is derived from a description of the extreme value distribution
- S = the score
- E = the expect value = the number of high-scoring segment pairs (HSPs) expected to occur with a score of at least S
- m, n = the lengths of two sequences
- λ, K = Karlin-Altschul statistics



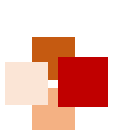


Some properties of equation $E = Kmn e^{-\lambda S}$



- The value of E decreases exponentially with increasing S (higher S values correspond to better alignments). Very high scores correspond to very low E values
- The S value for aligning a pair of random sequences must be **negative**. Otherwise, long random alignments would acquire great scores
- Parameter K describes the search space (database)
- For $E = 1$, one match with a similar score is expected to occur by chance. For a much larger or smaller database, you would expect E to vary accordingly





E-values and *p*-values (1)



- The **expect value** E is the number of alignments with scores greater than or equal to score S that are expected to occur by chance in a database search
- A ***p*-value** is a different way of representing the significance of an alignment

$$p = 1 - e^{-E}$$



E -values and p -values (2)



E values of about 1 to 10 are far easier to interpret than corresponding p values

Very small E values are very similar to p values

<u>E</u>	<u>p</u>
10	0.99995460
5	0.99326205
2	0.86466472
1	0.63212056
0.1	0.09516258 (about 0.1)
0.05	0.04877058 (about 0.05)
0.001	0.00099950 (about 0.001)
0.0001	0.00010000

- E -values are comparable to p -values, and are designed to be more convenient to interpret

